

Video 1: What Is AI-Assisted Coding?

The mental model, boundaries, and mindset shift every aspiring AI professional needs

Learning Objectives

By the end of this session, you should be able to:

- Explain AI-assisted coding and how it differs from traditional coding.
- Describe the mental model of vibe coding and AI code generation.
- Identify scenarios where AI-assisted coding is effective or limited.
- Recognize best practices for working with AI coding tools.
- Outline the role of tools like Visual Studio Code and GitHub Copilot.

Agenda

In this session, we'll discuss:

- AI-Assisted Coding: Concept and Workflow
- Traditional vs. AI-Assisted Development
- Mental Model: Vibe Coding and Roles
- Capabilities and Limitations of AI Coding
- Best Practices: Do's and Don'ts
- Tooling Overview: Visual Studio Code and GitHub Copilot

Understanding AI-Assisted Coding

A new way of working—not just a faster way of
typing

Traditional vs. AI-Assisted Coding

Traditional Coding

- You write every line of code yourself.
- Requires deep syntax and language knowledge.
- You debug manually, line by line.
- Slow to prototype new ideas.
- The code itself is the primary output.

AI-Assisted Coding

- You describe what you want. AI writes the code.
- Focus shifts from syntax to intent and direction.
- AI helps identify and explain errors.
- Rapid prototyping from plain English descriptions.
- The outcome is the primary output.

Key Shift: You become the Director. AI becomes the Implementer.

The Mental Model of Vibe Coding

How to think about AI so you stay in control

AI Is Not a Software Engineer—It's a Probabilistic Generator

Key Definition: AI coding assistants predict the most plausible next token. They do not reason about correctness.

What AI Actually Is

- An advanced autocomplete system
- A pattern matcher trained on billions of lines of code
- A probabilistic predictor: it picks what looks most plausible
- Optimized for coherence, not correctness

What AI Is Not

- A software engineer who understands your problem
- A reasoning system that checks for logical errors
- A reliable source of truth—plausible $\neq$ correct
- A replacement for your judgment and review

The Right Mental Model: The Confident but Forgetful Junior Dev

Think of AI as a junior developer who writes fast, has seen a lot of examples, but sometimes gets details wrong and forgets what you told them 10 minutes ago.

Writes Fast

- Generates code quickly.
- Great for boilerplate, standard patterns, and scaffolding.
- Speed is its strongest suit.

Has Seen a Lot

- Trained on millions of code examples.
- Knows common patterns extremely well, but only what it has seen before.

Forgets and Drifts

- Has a limited context window.
- Forgets earlier instructions.
- Makes assumptions without telling you.
- Needs constant guidance.

This mindset naturally leads you to: verify AI output, constrain its scope, and guide it more effectively.

The Mental Model: Three Roles You Play

Navigator

- Set the direction and goal.
- Break work into clear, small steps.
- Keep the big picture in focus.
- Know what "done" looks like.

Reviewer

- Inspect every AI-generated output.
- Ask: Does this do what I asked?
- Spot gaps, errors, or drift from intent.
- Stay curious, never passive.

Decision-Maker

- Accept, reject, or redirect AI output.
- Decide when to iterate vs. move on.
- Own the final quality of the work.
- You are still accountable, always.

All three roles work together. Neglect any one and the AI will drift, fail, or mislead you.

When Vibe Coding Works and When It Doesn't

Setting realistic expectations from day one

Where AI-Assisted Coding Shines

Prototyping and Exploration

Quickly test an idea or build a proof-of-concept without deep coding knowledge. Great for early experimentation.

Repetitive Boilerplate

Generating standard patterns: data loading, API calls, file handling, CSV parsing, formatting. AI handles these fluently.

Translating Logic to Code

You know what the logic should be. AI handles the syntax and structure. Turn plain English into working Python.

Explaining Unfamiliar Code

Ask AI to walk you through code you didn't write or don't fully understand. Like having a patient tutor on demand.

Where AI-Assisted Coding Falls Short

Highly Specialized Domains

Niche libraries, proprietary systems, or cutting-edge research methods the AI hasn't seen. Outputs will be confidently wrong.

Long or Complex Codebases

AI loses context across large projects. Risks inconsistency, duplication, and forgetting earlier decisions and constraints.

Security-Critical Code

AI can produce code that looks correct but has subtle vulnerabilities. Never trust AI-generated auth, encryption, or access control without deep review.

Novel Problem Solving

When there's no clear precedent, AI generates plausible-sounding but wrong answers. Originality is a human skill.

Rule of Thumb: The more context the AI lacks, the more oversight you need.

The Do's and Don'ts

Principles that separate effective vibe coders
from frustrated ones

Do's and Don'ts of AI-Assisted Coding

DO

- Verify every output the AI gives you.
- Work in small, focused steps—one task at a time.
- Ask the AI to explain code you don't understand.
- Treat AI as a collaborator, not an oracle.
- Test with inputs you can verify manually.
- Read generated code before running it in production.

DON'T

- Copy-paste AI code without reading it first.
- Ask for everything in one giant, monolithic prompt.
- Assume it is correct because it looks right.
- Use it for tasks where you cannot verify the output.
- Trust AI on security-critical or financial code without deep review.
- Confuse speed with quality; they are not the same.

Tooling Overview: VS Code + GitHub Copilot

VS Code

- Industry-standard code editor
- Free and open-source
- AI extensions plug in natively
- Works across Python and all major languages
- Integrated terminal for running code instantly

GitHub Copilot/Codex

- AI coding assistant built on OpenAI Codex
- Inline autocomplete as you type
- Chat interface for longer prompts and explanations
- Understands context across your open files
- Can write, explain, fix, and refactor code

Your Role in the Stack

- You write prompts, goals, and requirements.
- You review and accept or reject suggestions.
- You direct the iteration loop.
- You own the final quality check.
- You are the most important part of this stack.

Key Takeaways

01

You're the Director, Not the Typist

AI handles syntax and structure. Your job is to set clear goals, review output, and stay in control of the outcome.

02

Know the Sweet Spot

AI coding excels at prototyping, explaining, and repetitive tasks but needs human judgment for complex or sensitive work.

03

The Mental Model Matters

Think of yourself as navigator + reviewer + decision-maker. This mindset makes every other skill in this course click into place.

Next up: Video 2—Building the Skill: How to prompt iteratively and build real applications with AI

Summary

Here's a brief recap:

- AI-assisted coding is an approach where code is generated based on intent rather than being written line by line.
- Traditional and AI-assisted coding represent different development styles in terms of control, speed, and execution.
- The mental model of vibe coding defines how developers guide, review, and make decisions with AI.
- Capabilities and limitations describe where AI coding performs well and where it requires caution.
- Best practices establish guidelines for effectively and safely working with AI-generated code.
- Tools like Visual Studio Code and GitHub Copilot provide the environment for AI-assisted development.

Learning Outcomes

You should now be able to:

- Apply AI-assisted coding to simple development tasks.
- Evaluate AI-generated code for correctness and reliability.
- Select appropriate use cases for AI-assisted coding.
- Implement best practices while working with AI tools.
- Use Visual Studio Code and GitHub Copilot to complete coding tasks.